ESP32 JUPYTER INTERFACING UTILITIES

University of Alexandria, Faculty of Engineering,
Computer and Systems Engineering Department.
CSE343 Embeded Systems Spring 2026

ESP32 Simple Sketch

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:
  int x = 6;
  int y = 7;
  Serial.println(x + 2 * y);

}
```

ESP 32 Data Streaming Sketch

```
// Include I2S driver
#include <driver/i2s.h>

// Connections to INMP441 I2S microphone
#define I2S_WS 25  // LRCL
#define I2S_SD 33  // DOUT
#define I2S_SCK 32 // BCLK

// Use I2S Processor 0
#define I2S_PORT I2S_NUM_0

// Define input buffer length
#define bufferLen 300
int16_t sBuffer[bufferLen];

void i2s_install() {
  // Set up I2S Processor configuration
  const i2s_config_t i2s_config = {
    .mode = i2s_mode_t(I2S_MODE_MASTER | I2S_MODE_RX),
    .sample_rate = 16000,
    .bits_per_sample = i2s_bits_per_sample_t(16),
    .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
    .communication_format = i2s_comm_format_t(I2S_COMM_FORMAT_STAND_I2S),
    .intr_alloc_flags = 0,
    .dma_buf_count = 3,
    .dma_buf_len = bufferLen,
    .use_apll = false
  };

  i2s_driver_install(I2S_PORT, &i2s_config, 0, NULL);
}
```

ESP 32 Data Streaming Sketch

```
void i2s_setpin() {
  // Set I2S pin configuration
  const i2s_pin_config_t pin_config = {
    .bck_io_num = I2S_SCK,
    .ws_io_num = I2S_WS,
    .data_out_num = -1,
    .data_in_num = I2S_SD
  };

  i2s_set_pin(I2S_PORT, &pin_config);
}
```

ESP 32 Data Streaming Sketch

```
int samplesRead;

bool record = false;
bool commandRecv = false;

void setup() {
  Serial.begin(38400);
  while (!Serial);

  // Set up I2S
  i2s_install();
  i2s_setpin();
  i2s_start(I2S_PORT);

  delay(500);

  Serial.println("Welcome to the data streaming application using the Nano 33 BLE Sense\n");
}
```

ESP 32 Data Streaming Sketch

```
void loop() {

  while (Serial.available()) {
    char incomingByte = Serial.read();

    if (incomingByte == 'x') {
      record = !record;
      delay(500);
    }
  }

  // get Audio
  samplesRead = onPDMdata();
  //Serial.println(samplesRead);
  // display the audio if applicable
  if (samplesRead) {
    // print samples to serial plotter
    if (record) {
      for (int i = 0; i < samplesRead; i++) {
        Serial.println(sBuffer[i]);
      }
    }
    // clear read count
    samplesRead = 0;
  }
}
```

ESP 32 Data Streaming Sketch

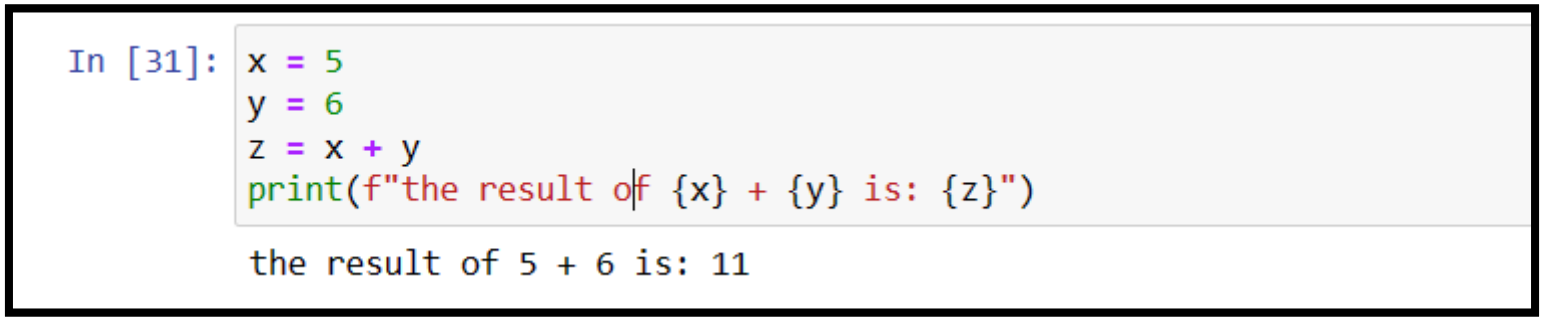
```
int onPDMdata() {

  // Get I2S data and place in data buffer
  size_t bytesIn = 0;
  samplesRead = 0;
  esp_err_t result = i2s_read(I2S_PORT, &sBuffer, bufferLen, &bytesIn, portMAX_DELAY);
  //Serial.println(bytesIn);
  if (result == ESP_OK)
  {
    // Read I2S data buffer
  samplesRead = bytesIn / 2;

  }
return samplesRead;
}
```

Jupyter

- To install Jupyter Visit:
  - https://jupyter.org/install

Jupyter


File
Edit
View
Insert
Cell
Kernel
Widgets
Help
Trusted
Python 3 (ipykernel)
Run
Code


In [31]:
x = 5
y = 6
z = x + y
print(f"the result of {x} + {y} is: {z}")
the result of 5 + 6 is: 11

Async In Jupyter

```
In [24]: import asyncio
         async def countAsync(task_no):
             for i in range(0,4):
                 print(f"Task {task_no}:" + str(i))
             return task_no * 6
         tasks = []
         for i in range(0,4):
             task = asyncio.create_task(countAsync(i+1))
             tasks.append(task)
         await asyncio.gather(*tasks)
```

```
Task 1:0
Task 1:1
Task 1:2
Task 1:3
Task 2:0
Task 2:1
Task 2:2
Task 2:3
Task 3:0
Task 3:1
Task 3:2
Task 3:3
Task 4:0
Task 4:1
Task 4:2
Task 4:3

Out[24]: [6, 12, 18, 24]
```

More Async Examples

- Visit https://stackoverflow.com/questions/50757497/simplest-async-await-example-possible-in-python

Setting Up

Importing Libraries

Include the necessary libraries for audio processing, communication with the Arduino, and TinyML inference in real time:

In [1]:
```
import asyncio
import time
from serial.tools import list_ports

import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np
import serial
from IPython.display import Audio, display, HTML

import tensorflow as tf
from tensorflow import keras


ports = list_ports.comports()
for port in ports:
    print(port)
```

```
COM9 - USB-SERIAL CH340 (COM9)
```

Initialization

Initialization

In this section, the serial connection with the Arduino is established, and the microphone is calibrated to normalize the input data within a specific range for improved processing and feature extraction.

In [2]:
```
SERIAL_PORT = 'COM9'
BAUD_RATE = 38400

def arduino_activate():
    try:
        arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(2)  # Allow time for connection to establish
    except serial.SerialException as e:
        print(f"Failed to connect to serial port {SERIAL_PORT}: \n{e}")
        return None

    command = input("Type 'x' to Activate Arduino: ")
    if command.lower() == 'x':
        arduino.write(command.encode('utf-8'))
        print("Success Activated!")
        print("Open the Serial Monitor to check if it is working, then close it.")
    return arduino


arduino = arduino_activate()
```

```
Type 'x' to Activate Arduino: x
Success Activated!
Open the Serial Monitor to check if it is working, then close it.
```

Initialization

In [4]:
```
def arduino_read(buffer, buffer_size, overlapping, norm=(None, None), max_attempts=10):
    buffer = np.roll(buffer, overlapping)
    num_data = buffer_size - overlapping
    for i in range(num_data):
        decoded_data = ''
        attempts = 0
        while decoded_data == '':
            arduino_data = arduino.readline()
            decoded_data = arduino_data[:len(arduino_data)].decode("utf-8").strip('\r\n')
            attempts += 1
            if attempts >= max_attempts:
                print('Fail to Retrieve Data...')
                break
        if norm[0] is not None:
            decoded_data = normalize(int(decoded_data), 1, -1, norm[0], norm[1])
        try:
            buffer[i+overlapping] = decoded_data
        except ValueError:
            print("Open the Serial Monitor to check if it is working. If it's not, press the reset button and rerun 'arduino_acti
    return buffer

def normalize(array, new_max, new_min, old_max, old_min):
    array = (((array - old_min) * (new_max - new_min)) / (old_max - old_min)) + new_min
    return array.astype(np.float16)
```

Initialization

```
# Calibrate the microphone
print("Calibrating: Please Speak to the Microphone")
time.sleep(1)
tuning_data = arduino_read(np.zeros(48000), 48000, 0)
TUNING_MAX, TUNING_MIN = (max(tuning_data), min(tuning_data))
print(f'Normalized signal from the range ({TUNING_MIN}, {TUNING_MAX}) to (-1, 1)')
```

```
Calibrating: Please Speak to the Microphone
Normalized signal from the range (-73.0, 80.0) to (-1, 1)
```

Running Inference

Running Inference

In this section, we set up asynchronous functions for MFCC features extraction. Then, we initialize the TensorFlow Lite interpreter, run emotion recognition inference, and send results back to the Arduino.

In [5]:
```
#Signal processing
BUFFER_SIZE = 24000
OVERLAPPED = 512

NUM_MFCC = 13
N_FFT = 2048
HOP_LENGTH = 512
SAMPLE_RATE = 16000
############################

EMOTIONS = ['neutral', 'happy', 'surprise', 'unpleasant']
COMMANDS = ['a', 'b', 'c', 'd']

# Manually Calibrate Sensitivity of Recognition
RECOG_MASK = np.array([1, 30000, 40000, 500])
```

Running Inference

```
async def tflite_process_data():
    data = np.zeros(BUFFER_SIZE)
    data = arduino_read(data, BUFFER_SIZE, OVERLAPPED, norm=(TUNING_MAX, TUNING_MIN))
    mfcc = librosa.feature.mfcc(y=data, sr=SAMPLE_RATE, n_mfcc=NUM_MFCC, n_fft=N_FFT, hop_length=HOP_LENGTH)
    features = np.array([mfcc.T], dtype=np.float32)

    # Model Input Shape = (None, None, 13)
    interpreter.set_tensor(interpreter.get_input_details()[0]['index'], features)
    interpreter.invoke()
    prediction = interpreter.get_tensor(interpreter.get_output_details()[0]['index'])[0]
    result = np.multiply(prediction, RECOG_MASK)
    emotion = EMOTIONS[np.argmax(result)]
    command = COMMANDS[np.argmax(result)]
    arduino.write(command.encode('utf-8'))
    print(result)
    print(f'Emotion: {emotion}\n')

async def tflite_run(rounds=10):
    tasks = []
    start_time = time.time()
    for turn in range(rounds):
        task = asyncio.create_task(tflite_process_data())
        tasks.append(task)
        time.sleep(0.6)
    await asyncio.gather(*tasks)
    display(HTML("<hr>"))
    print(f"Inference time for {turn+1} rounds: {time.time() - start_time} seconds")
```

Running Inference

```
interpreter = tf.lite.Interpreter(model_path="output/Models/SER_quant.tflite")
interpreter.allocate_tensors()
await tflite_run(rounds=10)
```

```
[0.9981817  0.25002718 0.76402117 0.89542108]
Emotion: neutral

[2.22413272e-01 4.73774783e+02 8.84739757e+02 3.69837880e+02]
Emotion: surprise

[ 0.96156669  2.46809701  8.57897685 19.06829886]
Emotion: unpleasant

[ 0.92424935 13.32177955 67.30651949 36.81198135]
Emotion: surprise

[0.99739873 0.16329179 1.48442705 1.27935165]
Emotion: surprise

[0.99600047 0.53795125 1.51739878 1.97183457]
Emotion: unpleasant
```